DS	**Session**	Queue	**Question Information**	Level 1 Challenge 59

Question description

On the last day of the semester, Shahid's students were talking and playing very loudly to the delight of the end of the semester. The principal of the college severely reprimanded Shahid. But he decided to engage them in a different activity without getting angry at the stu

So Shahid gave his students to solve the task such that, Circular Queue using Linked List and dequeue two elements from circular queue.

there is no memory waste while using Circular Queue, it is preferable than using a regular queue.

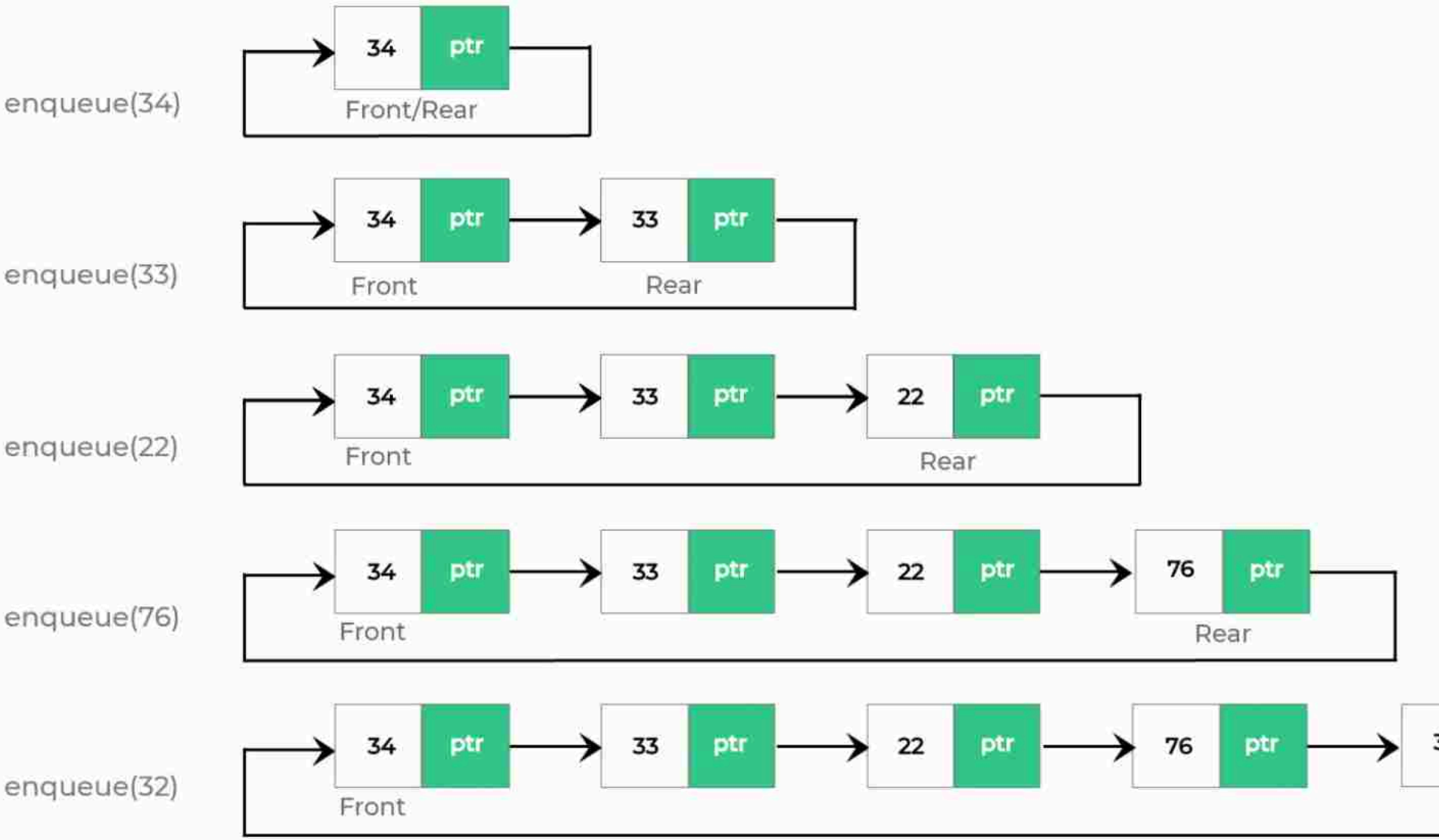
Because linked lists allow for dynamic memory allocation, they are simple to build.

Circular Queue implementation using linked list is identical to circular linked list except that circular Queue has two pointers front and back whereas circular linked list only has one pointer head.

Final Year students were lacking the idea to solve the problem.

Being an exciting youngster can you solve it?

Function Description



Removing the elements from Queue



Constraints:

0<size <100

0<data<1000

Input format:

First line indicates the size of the queue

Second line indicates the elements of the queue.

Output Format:

First line indicates the inserted elements in queue using linked list concept

second line indicates the one element dequeued from the queue using linked list concept.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5
9 1 5 3 5
```

EXPECTED OUTPUT

```
9 1 5 3 5
1 5 3 5
5 3 5
```

Test Case 2

INPUT (STDIN)

```
8
1 8 6 4 9 5 3 5
```

EXPECTED OUTPUT

```
1 8 6 4 9 5 3 5
8 6 4 9 5 3 5
6 4 9 5 3 5
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void enqueue(int d)
```

Test Case 2

KEYWORD

```
struct node* next;
```

Test Case 3

KEYWORD

```
n = (struct node*)malloc(sizeof(struct n
```

Test Case 4

KEYWORD

```
struct node *f = NULL;
```

Test Case 5

KEYWORD

```
struct node *r = NULL;
```

Test Case 6

KEYWORD

```
n->data = d;
```

Test Case 7

KEYWORD

```
n->next = NULL;
```

Test Case 8

KEYWORD

```
struct node* t;
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
3
```

Test Case 2

TOKEN COUNT

```
355
```

Test Case 3

NLOC

```
75
```

Code

solution.c

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node* next;
};

struct node *f = NULL;
struct node *r = NULL;

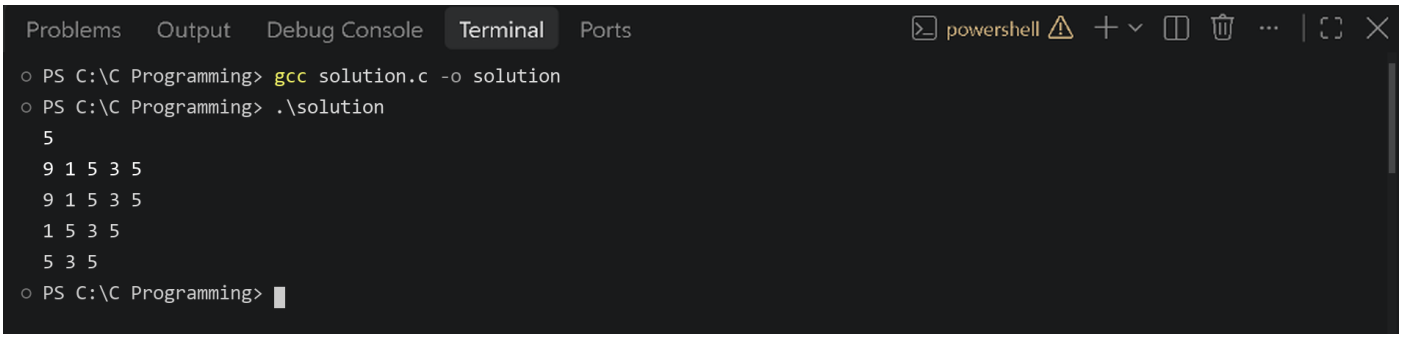
void enqueue(int d) {
    struct node* n = (struct node*)malloc(sizeof(struct node));
    n->data = d;
    n->next = NULL;
    if (f == NULL) {
        f = r = n;
        r->next = f;
    } else {
        r->next = n;
        r = n;
        r->next = f;
    }
}

void dequeue() {
    if (f == NULL) return;
    if (f == r) {
        free(f);
        f = r = NULL;
    } else {
        struct node* t = f;
        f = f->next;
        r->next = f;
        free(t);
    }
}

void display() {
    if (f == NULL) return;
    struct node* t = f;
    do {
        printf("%d%s", t->data, (t->next == f) ? "" : " ");
        t = t->next;
    } while (t != f);
    printf("\n");
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    for (int i = 0; i < n; i++) {
        int x; scanf("%d", &x);
        enqueue(x);
    }
    display();
    dequeue();
    display();
    dequeue();
    display();
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
5
9 1 5 3 5
9 1 5 3 5
1 5 3 5
5 3 5
PS C:\C Programming>